TECHNICAL REFERENCE

Gateway Guard Connection architecture

Languages, installer contents, and how a ChatGPT prompt is allowed or blocked

1. Languages used in this product

The product is three runtimes. The server and the admin UI stay on the Gateway host. The desktop Guard is a separate program installed on each employee machine.

Gateway server

- Go 1.26. The HTTP API, admin UI host, Browser AI target list, Prompt Logs, and Guard rules live here.
- Plugins are also Go: guardrails, governance, prompts, logging, semantic cache, telemetry, OpenTelemetry.
- HTTP stack is fasthttp. Data store is SQLite through the config store. Sessions use JWT.

Admin and workspace UI

- TypeScript and React, built with Vite. The production UI is compiled into the Go binary and served by the Gateway.
- This is the console where admins set Guard rules, Target Websites, and read Prompt Logs.

Gateway Guard (desktop)

- Python 3. The agent and the local proxy are Python.
- The local proxy is a mitmproxy-style HTTPS intercept. It sees browser traffic only for configured AI sites.
- Windows setup script is Inno Setup 6. It wraps the program into Gateway_Guard_Setup.exe.
- The employee binary is built with PyInstaller into Gateway_Guard.exe. An optional Nuitka path compiles the same agent to a native binary.
- Config is JSON (gateway_guard_config.json). Proxy logic shipped to employees is stored as encrypted containers, not as loose .py files.

2. What is inside the Guard

Product name: Gateway Guard. Windows program name: Gateway_Guard.exe. Installer name: Gateway_Guard_Setup.exe. Current installer version in the Inno script: 1.1.23. Install folder: %LOCALAPPDATA%\Programs\Gateway\Guard.

Gateway_Guard_Setup.exe

This file is the Inno Setup installer, not the proxy itself. It copies the application, registers Gateway Guard to start at Windows login, writes the uninstall entry, and can start the Guard immediately. A gateway_guard_config.json placed next to the setup file overrides the config baked into the build.

Gateway_Guard.exe

PyInstaller packs the Python agent into this one executable. At runtime it contains:

- Agent: PAC file writer, local proxy listener, TLS certificate install, heartbeat, and auto-update against the Gateway.
- Encrypted proxy bundle: browser AI proxy plus the parts listed below. Plain Python sources are stripped from the release bundle.
- gateway_guard_config.json: Gateway URL, proxy listen address, and PAC settings.
- Runtime files written on the PC: proxy.pac and a local log. These are not source code.

Proxy parts inside the bundle

- config_caches_rules.py: loads Guard rules and decides Block, Redact, Warn, or Allowed.
- helpers_prompts.py: pulls the typed prompt out of ChatGPT, Claude, Gemini, DeepSeek, and other site payloads.
- uploads_detect.py: detects file and voice uploads and caches the bytes until the user presses Send.
- file_policy.py: extracts text from PDF, Office, images, and voice, then runs rules per file.
- extract_office_backend.py: reads Office document bodies.
- responses_inject.py: writes the block or redact notice back into the site response so the chat page shows it.
- responses_addon.py: the live hook. Every monitored request and WebSocket frame passes through here.

The Go server and the React console are not inside the exe. The exe only talks to them over HTTPS.

3. Connection structure

Example: the employee opens ChatGPT in the browser and sends a prompt, a file, or a voice note. The same path is used for Claude, Gemini, DeepSeek, and any other host listed as a Target Website.

1. Browser opens the AI site

Chrome or Edge opens chatgpt.com. Gateway Guard is already running and has set a PAC file so that site is sent to the local proxy, not straight to the internet.

2. User sends prompt, file, or voice

A typed message, an attachment, or a voice clip leaves the browser as normal HTTPS or WebSocket traffic.

3. Local proxy receives it

Gateway_Guard.exe terminates TLS with its local certificate, reads the request, and keeps the original bytes.

4. File and voice wait for Send

An upload is cached on the PC. The proxy does not score the file on the upload request itself. Scoring starts when the chat Send is detected.

5. Extract

The proxy reads the typed prompt. For a file it extracts text (PDF, Office, image OCR, archives where supported). For voice it uses the transcript the site or the proxy already produced.

6. Predict

Extracted text is matched against Guard rules synced from the Gateway. The result is one action: Allowed, Warn, Redact, or Block.

7. Allow, warn, redact, or block

Allowed continues the original request to ChatGPT. The other actions are below.

8. Prompt Log

The decision, the site, and the prompt or file name are posted to the Gateway and appear in Prompt Logs.

What each decision does

- Allowed: no rule matched. The original prompt, file, or voice continues to the AI site. The model answers normally. The log row is Allowed.
- Warn: the request is allowed, and a warning is injected into the chat UI so the user sees the rule notice.
- Redact: matching text is removed or replaced, the cleaned request is allowed through, and a redact notice is injected into the page when the site response can be rewritten.
- Block: the original prompt is not delivered as a normal model turn. The proxy returns a local response that the site

UI renders as the assistant message, so the user sees the block reason inside ChatGPT, Claude, Gemini, or DeepSeek. A repeat of the same blocked text is dropped without a second bubble.

- Site block: if the whole target is denied, the proxy blocks the host before a prompt is scored and logs a site-blocked row.

Why some files are not scored

The proxy keeps a file only when the Send names it, or the bytes have a known document or media header, or the body is plain text. Unnamed binary chunks, uploads older than the current send burst (about 60 seconds), and more than 128 files on one send are dropped. The same file name is logged once per 20 seconds. Files over 20 MB are not stored for the temporary view.

4. One-line path

Browser (ChatGPT / Claude / Gemini / DeepSeek)
-> PAC -> Gateway_Guard.exe local proxy
-> extract prompt / file / voice -> Guard rules
-> Allowed | Warn | Redact | Block -> Prompt Log on the Gateway

The model provider sees the request only after the proxy allows it. A block is answered by the Guard on the employee PC. Rules and logs stay on the Gateway server.